

Relatório Técnico e Memorial de Entrega do Software

Projeto de Software: Portal de Solicitações Internas

Bit Soluções

2026



SUMÁRIO

1. ESCOPO GERAL E DELIMITAÇÃO	3
2. TECNOLOGIAS UTILIZADAS (STACK)	3
2.1. FRONT-END	3
2.1.2. SELEÇÃO E JUSTIFICATIVA DA STACK: FRONT-END	4
2.2. BACK-END	4
2.2.2. SELEÇÃO E JUSTIFICATIVA DA STACK: BACK-END	5
3. ARQUITETURA E INTEGRAÇÃO DA APLICAÇÃO	5
3.1 ARQUITETURA EM CAMADAS DE BAIXO ACOMPLAMENTO	5
3.2 FLUXO INTERNO DO BACK-END	5
4. BANCO DE DADOS E PERSISTÊNCIA	6
5. SEGURANÇA E AUTENTICAÇÃO	7
6. APIS RESTFUL E INTEGRAÇÃO	7
6.2. DOCUMENTAÇÃO DA API	8
7. CONTAINERIZAÇÃO E DEPLOY	8
8. AUTOAVALIAÇÃO DE DESEMPENHO	9
9. CONCLUSÃO	9

1. ESCOPO GERAL E DELIMITAÇÃO

O projeto de software, Portal de Solicitações Internas, consiste no desenvolvimento de uma aplicação web voltada para o gerenciamento e acompanhamento de solicitações para demandas internas de uma empresa prestadora de serviços. O sistema centraliza solicitações divididas entre vários setores: Recursos Humanos (RH), Tecnologia (TI), Infraestrutura, Finanças e Compras.

A interface foi projetada inicialmente para o uso no layout de computadores e notebooks (desktops), contando com design responsivo e portabilidade nativa para se adaptar automaticamente a diferentes resoluções de telas, incluindo dispositivos móveis (celulares e tablets).

O software oferece aos usuários as operações fundamentais de gerenciar solicitações com operações fundamentais de gerenciamento (CRUD), permitindo a criação, visualização, edição e exclusão de registros de solicitações. Além de disponibilizar a visualização de um dashboard interativo contendo indicadores e gráficos informativos para consultas rápidas dos registros em tempo real, no qual é possível visualizar e interagir dinamicamente. A navegação do sistema é estruturada por meio de um menu lateral principal (side menu), composto pelas páginas:

- I. Dashboard: Exibe as principais métricas e indicadores gerais das solicitações.
- II. Nova Solicitação: Formulário padronizado para abertura de novos registros para demandas internas.
- III. Gerenciar Solicitações: Seção de gerenciamento com tabela de listagem completa, com suporte a pesquisa, filtros por categoria, status e período. Além de ações de visualização, edição e exclusão.
- IV. Perfil: Visualização das informações pessoais e de acesso do colaborador.

2. TECNOLOGIAS UTILIZADAS (STACK)

2.1. FRONT-END

A interface de apresentação foi desenvolvida utilizando um ecossistema de tecnologias modernas para o desenvolvimento web: Next.js, Tailwind CSS, TypeScript e Lucide Icons. A escolha deste conjunto de tecnologias visou garantir a alta usabilidade, excelente performance de renderização e uma estética de modernização, corporativismo fluido e eficiência profissional para o software em sua organização de layout.

Antes do início da codificação, foram levados em consideração os princípios essenciais de Experiência de Usuário (UX) e Design de Interface (UI) para assegurar uma navegação fluida e intuitiva. Adotou uma paleta de cores pautada em cores neutras (branco, preto e variações de tonalidades de cinza) combinada com um tom primário de cor verde corporativo (Hexadecimal: **#176b45**), utilizado para destacar elementos de ações, botões e destacar elementos.

2.1.2. SELEÇÃO E JUSTIFICATIVA DA STACK: FRONT-END

- Next.js: o framework baseado em React foi escolhido por motivo de proporcionar alto desempenho pelo seu sistema de roteamento nativo baseado em pastas (App Router). Recursos nativos, como as otimizações de tags `<Image/>` e `<Link/>` garantem o rápido carregamento em milissegundos (ms), com transições instantâneas entre as páginas sem a necessidade de recarregamento completo da aplicação.
- TypeScript: optou pela utilização da linguagem de programação TypeScript em detrimento do JavaScript puro devido à checagem estática de tipagem. A camada de compilação e transpilação traz segurança e previsibilidade durante o fluxo de desenvolvimento, prevenindo erros e exceções não tratadas de tipagem em tempo de execução.
- Tailwind CSS e Lucide Icons: o framework Tailwind CSS permitiu a estilização ágil e padronizada dos componentes, mantendo o princípio de clean code e com pacotes de estilos otimizados. Ao adicionar a interação com a biblioteca Lucide Icons, enriquece a navegação com ícones vetoriais leves, facilitando o processo de identificação rápida de ações e operações.

2.2. BACK-END

O back-end da aplicação foi desenvolvido na linguagem de programação Java, versão 21 (LTS), utilizando o framework Spring Boot 4.0.8, iniciado via Spring Initializer, com gerenciamento de bibliotecas e dependências e compilação por Apache Maven. A escolha desta arquitetura visa fornecer uma API RESTful robusta de alta performance, desacoplada da camada visual e estruturada sob o padrão MVC em camadas específicas de Controller, Service, Repository e DTOs.

Configurações utilizadas:

- Linguagem: Java 21 LTS.
- Framework: Spring Boot 4.0.8.
- Empacotamento: JAR.
- Gerenciador: Apache Maven.

2.2.2. SELEÇÃO E JUSTIFICATIVA DA STACK: BACK-END

- Java 21 + Spring Boot: o ecossistema Java com Spring Boot é referência no mercado para aplicações escaláveis e robustas em corporações. A versão 21 do Java garante estabilidade a longo prazo, alta performance e segurança em execuções. Além de usar recursos modernos da JVM. O framework Spring Boot simplifica a criação de APIs REST ao fornecer autoconfiguração, reduzindo o tempo de configuração inicial e ao longo do desenvolvimento do setup.
- Spring Data JPA e Hibernate, e PostgreSQL: atuam na camada de abstração de persistência dos dados, por meio do ORM, eliminando a necessidade de escrita manual de queries SQL para operações de CRUD. O PostgreSQL Driver é uma biblioteca essencial de conectividade JDBC.
- Validation (Jakarta Validation) e Lombok: a Validation é utilizada no tratamento das requisições de entradas. Por meio de anotações nos DTOs, garantindo a integridade. O Lombok é utilizado para reduzir o código, por meio de anotações em tempo de compilação, injetando automaticamente métodos getters e setters.

3. ARQUITETURA E INTEGRAÇÃO DA APLICAÇÃO

3.1 ARQUITETURA EM CAMADAS DE BAIXO ACOMPLAMENTO

A solução foi projetada sob estrutura desacoplada ou de baixo acoplamento, separando claramente as responsabilidades de forma clara por classes. A primeira camada foi a de front-end em Next.js e a camada de serviços e regras de negócios no Java Spring Boot. A comunicação entre os dois ecossistemas ocorre de forma assíncrona por requisições HTTP/RESTful com tráfego em arquivo em formato . JSON. No back-end foi adotado o padrão de arquitetura em camadas, garantindo os princípios SOLID, principalmente o de responsabilidade única.

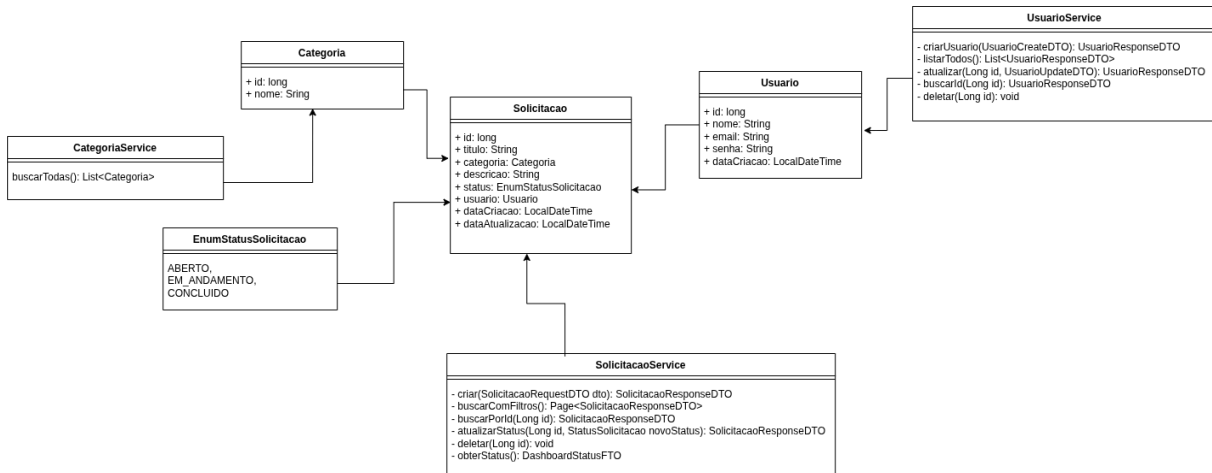
3.2 FLUXO INTERNO DO BACK-END

O back-end da aplicação é organizado em camadas, seguindo uma estrutura que separa as responsabilidades de cada parte do sistema. O fluxo de uma requisição passa pelas seguintes camadas:

- Models: representam as entidades do sistema e seu mapeamento para as tabelas do banco de dados, definindo os atributos e relacionamentos entre os dados.
- Repositories: são responsáveis pelo acesso e persistência dos dados no banco de dados. Utilizam o Spring Data JPA para realizar as operações.

- DTOs: são utilizados para transportar dados entre as diferentes camadas da aplicação, principalmente entre o back-end e o cliente.
- Services: concentram as regras de negócios da aplicação. Essa camada processa as informações recebidas.
- Controllers: responsáveis por receber as requisições HTTP da API e redirecioná-las para os services e retornar a resposta.

Figura 1: Diagrama de Classes UML reduzido aos models, services e enums.

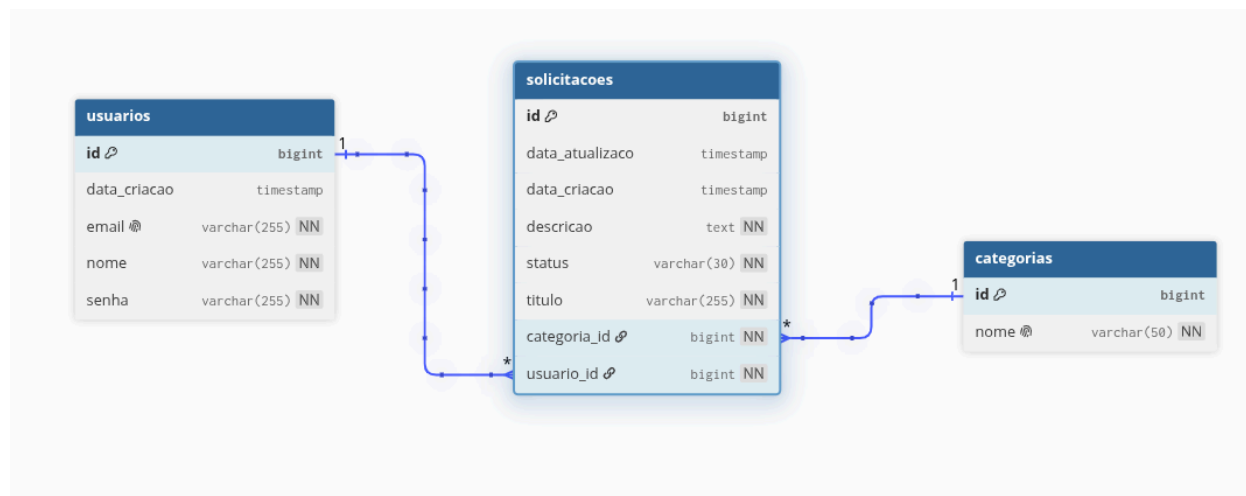


4. BANCO DE DADOS E PERSISTÊNCIA

O sistema utiliza o PostgreSQL como sistema gerenciador de banco de dados relacional, sendo responsável por armazenar a persistência das informações utilizadas pela aplicação, como usuários, categorias e solicitações. A persistência dos dados no back-end é realizada por meio do Spring Data JPA, utilizando o Hibernate como implementação ORM. Assim, permite que as entidades Java sejam mapeadas corretamente.

- Usuários: armazena os dados dos usuários cadastrados, como nome, e-mail, senha e data.
- Solicitações: armazena as demandas registradas no sistema, contendo informações como título, descrição, status, categoria, usuário responsável e datas.
- Categorias: armazena as categorias, como TI, RH, Compras, Financeiro e Infraestrutura.

Figura 2: Diagrama Entidade-Relacionamento do banco de dados.



5. SEGURANÇA E AUTENTICAÇÃO

A segurança da aplicação foi implementada com o objetivo de proteger o acesso às funcionalidades do sistema e garantir que as operações realizadas na API sejam executadas por usuários autenticados. Utiliza recursos de segurança do Spring Security, enquanto o front-end mantém o estado autenticado por meio de um token JWT armazenado em cookie HTTP-only. A comunicação entre as camadas ocorre pelo padrão de fluxo de requisições HTTP.

O token possui um período de validade definido pela aplicação e é assinado utilizando uma chave secreta. Nas requisições destinadas a setores protegidos, o token é enviado em cabeçalho HTTP.

As senhas dos usuários não são armazenadas em texto puro no banco de dados. Para realizar esse procedimento, foi preferível o algoritmo BCrypt, disponibilizado pelo PasswordEncoder do Spring Security. No cadastro, a senha informada é transformada em hash antes de ser armazenada no banco e posteriormente comparada para operações de autenticação com a verificação hash do BCrypt.

6. APIS RESTFUL E INTEGRAÇÃO

A comunicação entre o front-end e o back-end da aplicação é realizada por meio de uma API REST, desenvolvida com Spring Boot. Essa abordagem permite manter as camadas de apresentação e de regras de negócio desacopladas. A API utiliza o formato JSON para troca de informações entre o cliente e o servidor, seguindo os princípios de uma arquitetura REST. Os

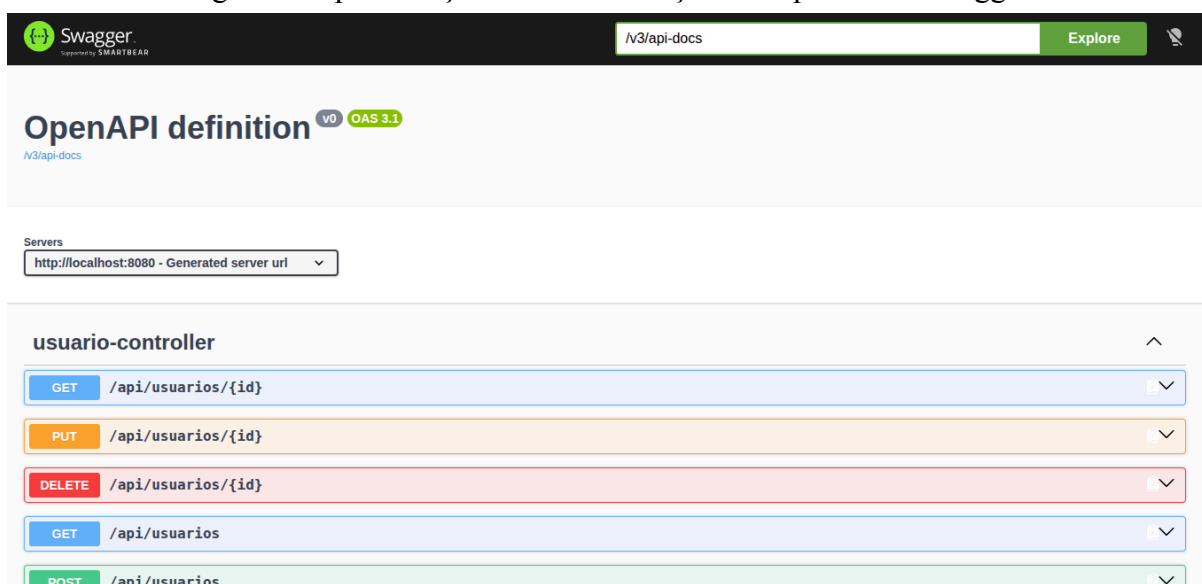
recursos são organizados de acordo com as principais entidades da aplicação, como usuários, categorias e solicitações.

Para controlar os dados recebidos e enviados pela API, o back-end utiliza DTOs. Essa abordagem permite definir quais informações devem ser recebidas ou retornadas em cada operação, evitando a exposição direta das entidades de persistência.

6.2. DOCUMENTAÇÃO DA API

A API é documentada utilizando OpenAPI/Swagger, permitindo visualizar os endpoints disponíveis, seus métodos HTTP, parâmetros e estruturas de requisição e resposta. A documentação também possibilita realizar testes diretamente pela interface do Swagger, facilitando a validação dos endpoints durante o desenvolvimento e servindo como referência técnica para o consumo da API.

Figura 3: representação da documentação e endpoints do Swagger.



7. CONTAINERIZAÇÃO E DEPLOY

A aplicação foi preparada para execução em ambiente containerizado, utilizando Docker e Docker Compose. A containerização foi adotada com o objetivo de padronizar o ambiente de execução e facilitar a inicialização integrada dos principais componentes do sistema. A solução é composta por três serviços principais: front-end, desenvolvido com Next.js; back-end,

desenvolvido com Spring Boot; e banco de dados, utilizando PostgreSQL. Cada serviço é executado de forma independente em seu próprio container.

O projeto possui arquivos Dockerfile específicos para o front-end e para o back-end, responsáveis por definir os ambientes necessários para construção e execução das aplicações. No front-end, o Dockerfile utiliza uma imagem baseada em Node.js para instalar as dependências do projeto, realizar o processo de build e iniciar a aplicação Next.js. No back-end, o Dockerfile utiliza o Java 21 para realizar a compilação da aplicação utilizando o Maven e executar o arquivo JAR gerado pelo Spring Boot. O PostgreSQL utiliza uma imagem oficial disponibilizada para Docker, sendo configurado diretamente no arquivo docker-compose.yml.

- Front-end: aplicação web desenvolvida em Next.js, disponibilizada na porta 3000.
- Back-end: API REST desenvolvida em Spring Boot, disponibilizada na porta 8080.
- PostgreSQL: banco de dados PostgreSQL, utilizado pelo back-end para persistência das informações.

8. AUTOAVALIAÇÃO DE DESEMPENHO

Embora a aplicação consiga cobrir o seu escopo delimitado e atenda aos requisitos funcionais de forma eficiente e eficaz. A autoavaliação do projeto permitiu identificar limitações conceituais e oportunidades de melhorias, contando com algumas fragilidades não exploradas de acordo com o documento de mini-projeto:

- Remoção de usuários para solicitações no endpoint (“/usuarios/{id}”) foi implementada como exclusão física, hard-delete, removendo o registro diretamente do banco de dados. Em sistemas atuais, essa abordagem apresenta riscos à integridade e ao histórico de consultas auditorais. Caso o usuário seja excluído, perde a rastreabilidade do solicitante original que abriu o chamado. Como melhoria, é recomendável a adoção da técnica de exclusão lógica, soft-delete, para ocultar o registro nas telas, sem comprometer o histórico no banco.
- O sistema não conta com gestão de perfis de acesso, nem módulo administrativo para cadastrar novos colaboradores, mas não conta com módulo de gestão. Em cenário de produção, seria essencial implementar um controle de acesso baseado em papéis, separando a visualização e operações por permissões de usuários.

9. CONCLUSÃO

O desenvolvimento do Portal de Solicitações Internas permitiu a implementação de uma solução web destinada ao gerenciamento de solicitações internas, contemplando funcionalidades de cadastro, consulta, atualização e exclusão de demandas, além do gerenciamento de usuários, categorias e autenticação.

A aplicação foi desenvolvida utilizando uma arquitetura separada entre front-end e back-end, permitindo a divisão das responsabilidades e a comunicação por meio de uma API RESTful. A utilização do Next.js no front-end e do Spring Boot no back-end possibilitou a construção de uma solução organizada em camadas, com separação entre apresentação, regras de negócio e persistência de dados.

Dessa forma, o projeto atende ao escopo inicialmente definido e demonstra a aplicação prática de conceitos de desenvolvimento web, arquitetura de software, persistência de dados, segurança, APIs RESTful e containerização em uma solução integrada para gerenciamento de solicitações internas.